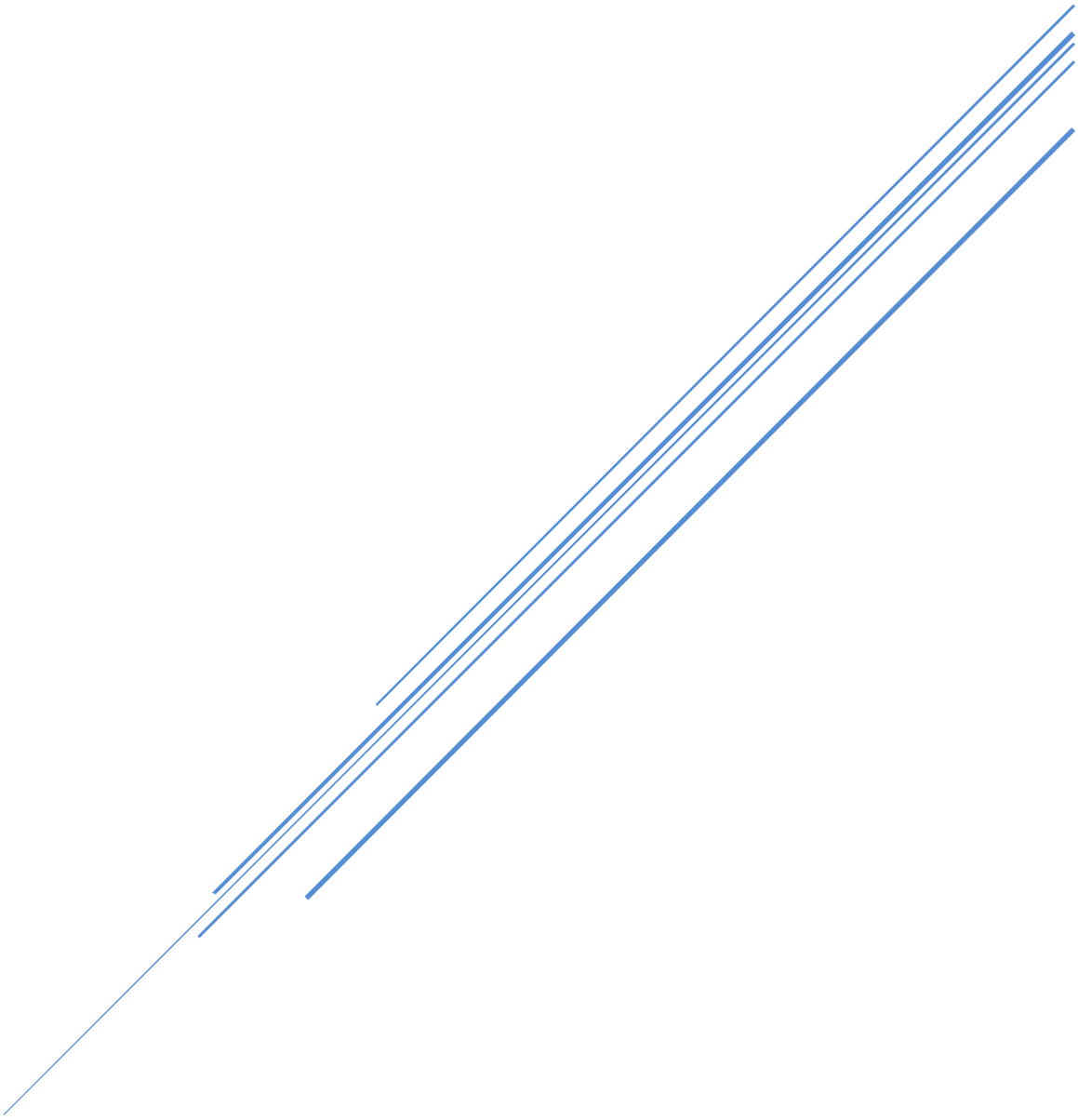
LABORATORIO 1.1

Línea base de disponibilidad



Mora Monje Kevin
05/11/2026

1. Objetivo

Medir el comportamiento inicial del microservicio catalog-service mediante solicitudes HTTP al endpoint /v1/products. Se registran los códigos de respuesta, los tiempos de respuesta y el contenido devuelto, para establecer una línea base que permita comparar el servicio después de aplicar mecanismos de resiliencia.

2. Entorno y procedimiento

- Sistema operativo: Windows, utilizando PowerShell.
- Servicio: catalog-service, ejecutado en modo desarrollo con Quarkus.
- Dirección local: http://localhost:8080
- Endpoint evaluado: GET /v1/products
- Herramienta de medición: script PowerShell test-availability.ps1.
- Cantidad de solicitudes: 200.
- Archivo de resultados: availability-baseline.txt.

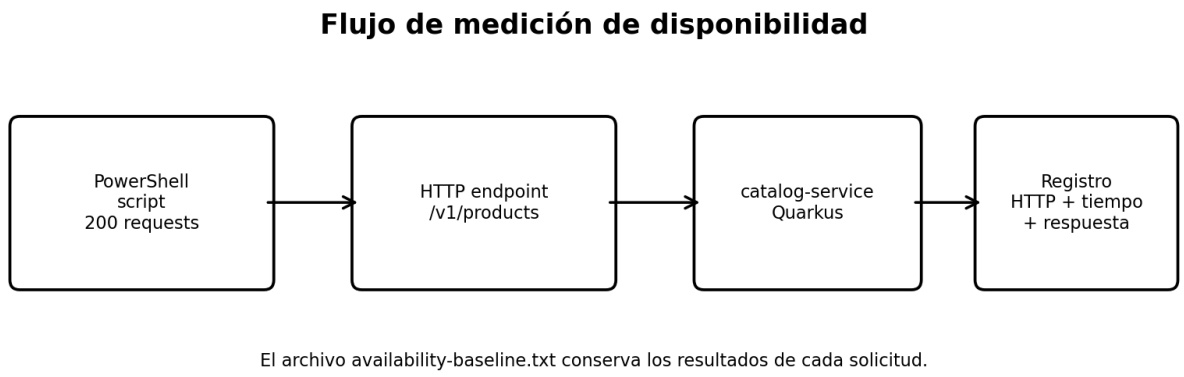


Figura 1. Flujo utilizado para ejecutar y registrar la prueba.

3. Verificación inicial de endpoints

Antes de ejecutar la prueba de carga, se verificaron los endpoints de salud y el endpoint principal. Las consultas realizadas devolvieron HTTP 200 y estado UP.

Endpoint	Resultado observado	Interpretación
/health	HTTP 200; status UP	Comprobación general correcta.
/ready	HTTP 200; status UP	Servicio reportado como listo.
/live	HTTP 200; status UP	Proceso activo.
/v1/products	HTTP 200; SUCCESS	Endpoint de productos responde.

4. Resultados de disponibilidad

Se ejecutaron 200 solicitudes. El resumen obtenido en PowerShell fue:

Métrica	Resultado
Respuestas SUCCESS	138
Respuestas HTTP 200	138

Respuestas distintas de HTTP 200	62
Solicitudes totales	200
Disponibilidad observada	69%
Tasa de respuestas distintas de HTTP 200	31%

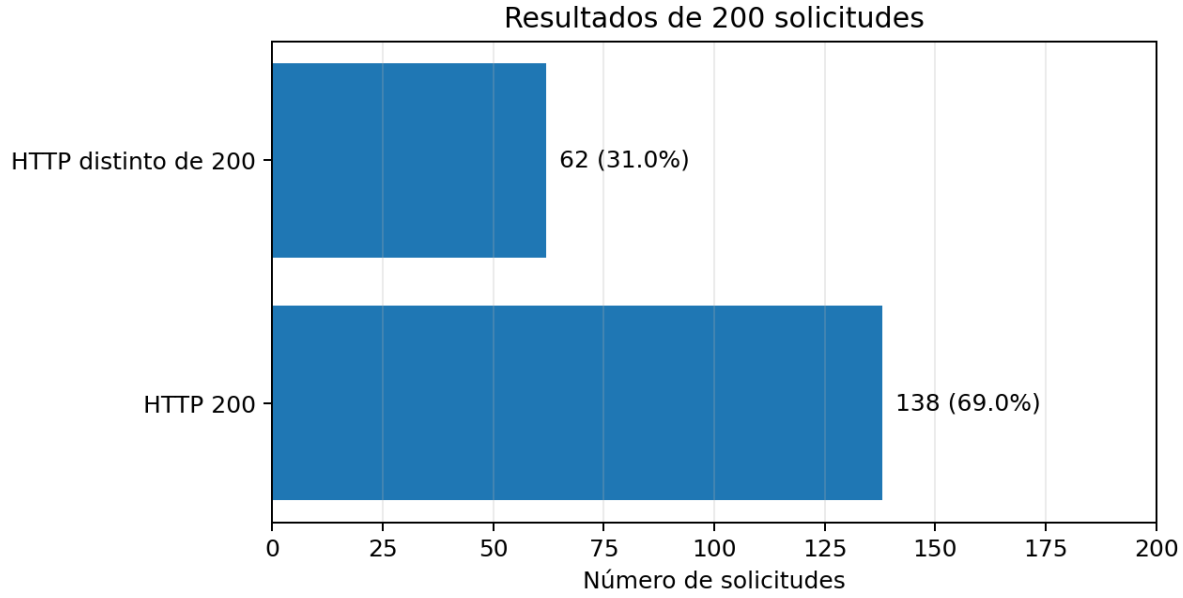


Figura 2. Distribución de respuestas de la prueba.

Cálculo: disponibilidad observada = (respuestas HTTP 200 / solicitudes totales) × 100 = (138 / 200) × 100 = 69%.

Interpretación: 138 solicitudes finalizaron con HTTP 200 y 62 devolvieron un código diferente de 200. En esta ejecución, la disponibilidad observada fue del 69%.

Evidencia 3. Salida final del script con 138 respuestas exitosas, 62 respuestas distintas de 200 y 69%.

5. Resultados de rendimiento

A partir de los tiempos registrados en availability-baseline.txt se obtuvieron los siguientes valores:

Métrica	Resultado
Solicitudes medidas	200
Tiempo promedio	0.549 segundos
Tiempo mínimo	0.012 segundos
Tiempo máximo	3.143 segundos
Solicitudes con tiempo superior a 1 segundo	35 (17.5%)

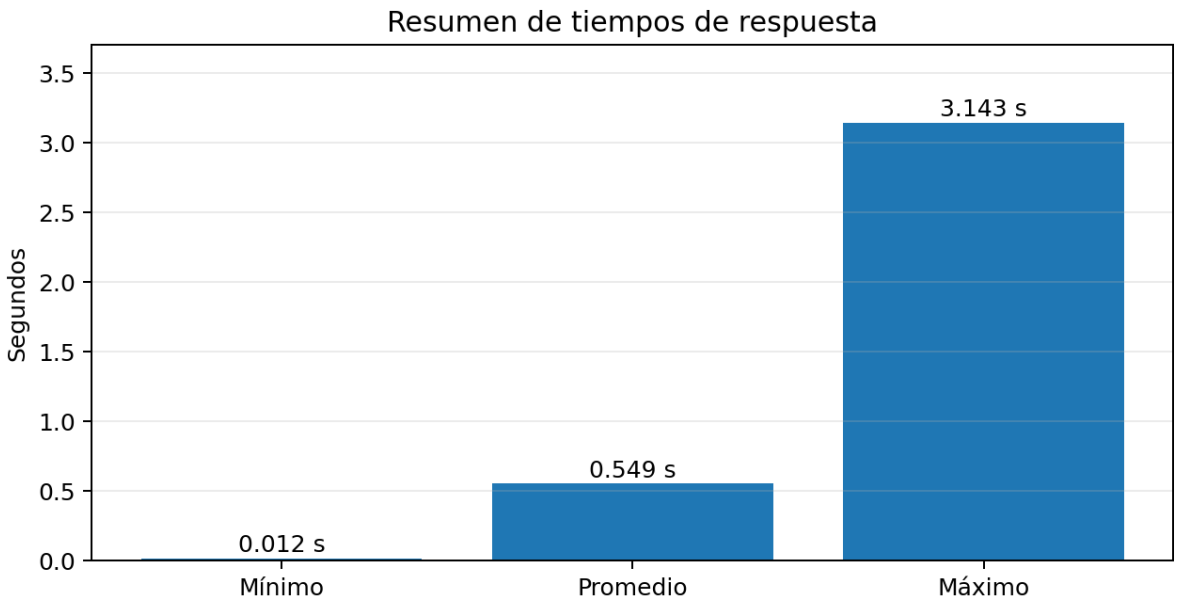
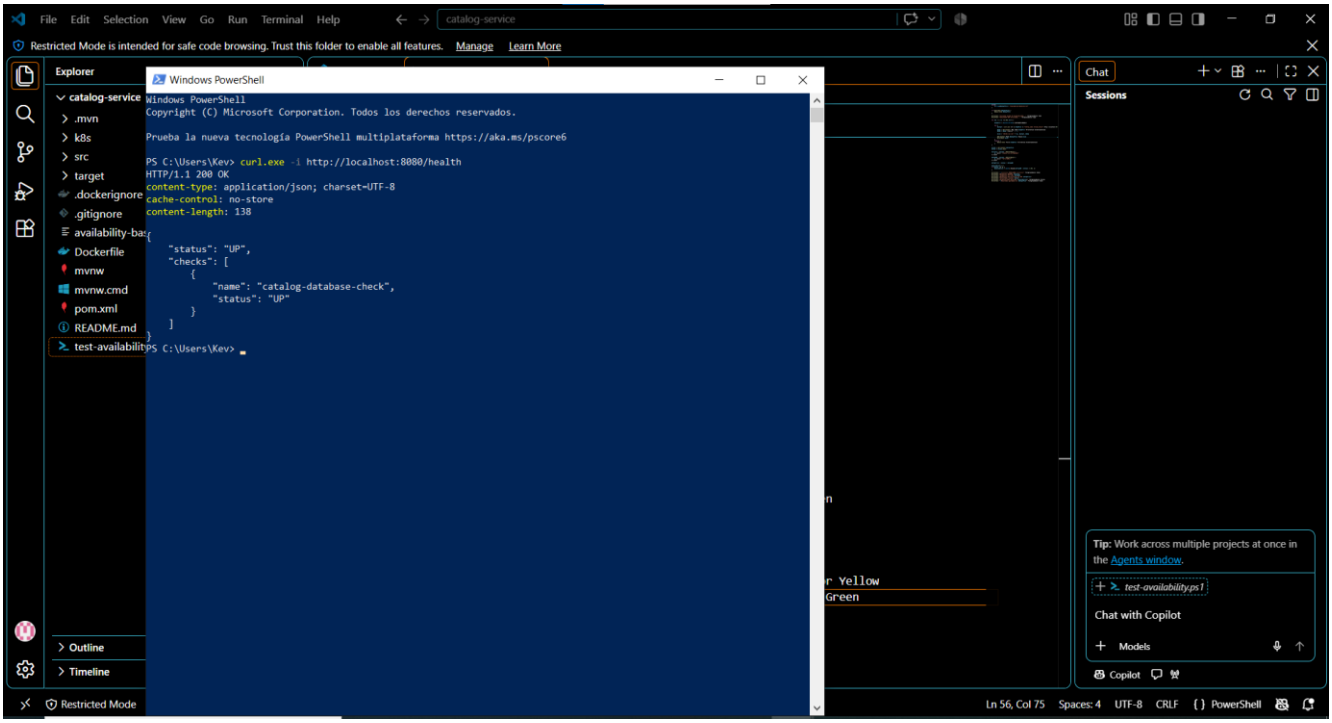


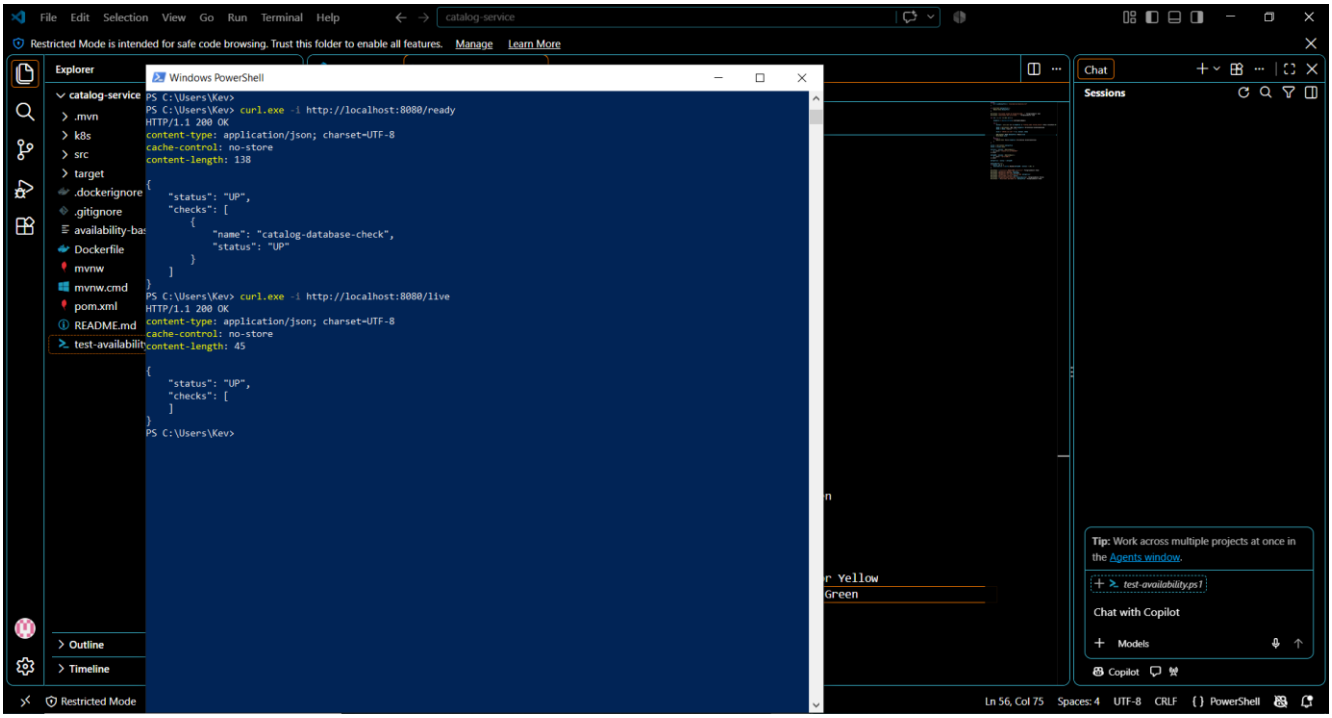
Figura 3. Comparación del tiempo mínimo, promedio y máximo.

Interpretación: el tiempo promedio fue de 0.549 segundos, aunque se observaron respuestas de hasta 3.143 segundos. Un total de 35 solicitudes superó un segundo. Una respuesta lenta no necesariamente representa un error HTTP: por ejemplo, entre los primeros registros hubo una solicitud que tardó 3.024 segundos y aun así devolvió HTTP 200.

Evidencia 4. Salida de PowerShell con promedio, mínimo, máximo y solicitudes mayores a 1 segundo.



Evidencia 5. Primeras líneas de availability-baseline.txt donde se observe el código HTTP y el tiempo.



6. Análisis general

La prueba muestra que el microservicio respondió correctamente a 138 de las 200 solicitudes realizadas, lo que equivale a una disponibilidad observada del 69%. Las 62 respuestas restantes tuvieron un código HTTP distinto de 200. En rendimiento, el tiempo promedio fue de 0.549 segundos, con un máximo de 3.143 segundos y 35 solicitudes por encima de un segundo.

Estos resultados constituyen la línea base del servicio. Permiten identificar tanto respuestas fallidas como variaciones de latencia. La disponibilidad y el rendimiento son indicadores relacionados, pero distintos: una respuesta puede ser lenta y aun así ser exitosa.

7. Conclusión

Se completó la medición inicial de catalog-service mediante 200 solicitudes al endpoint /v1/products. La disponibilidad observada fue del 69%, con 138 respuestas HTTP 200 y 62 respuestas diferentes de 200. El tiempo promedio de respuesta fue de 0.549 segundos y el máximo de 3.143 segundos. A partir de esta línea base se podrán comparar los cambios que se obtengan al implementar mecanismos de resiliencia como Timeout, Retry, Fallback y Circuit Breaker. El propósito de estos mecanismos será mejorar el manejo de fallos, controlar las esperas y evitar que los problemas de una dependencia afecten de manera prolongada al servicio.